

实验报告 / Experiment Report — Evolving Quant Agent (QEA) v0

日期 / Date: 2026-06-09 · 项目 / Project: Evolving Quant Agent v0 · 仓库 / Repo: github.com/Wrigggy/evolving-quant-agent

一句话 / TL;DR: v0 的机制成立、但还没证出 quant 上的"价值"。离线 mock 下「自动改造 agent 外挂(harness)→ 验证 → 不行就回滚」这套闭环四项信号全亮；接到真实 GDPval 财务任务后闭环能端到端跑通，但没观察到提升 —— 主要被软评分噪声卡住，加上 worker 目前只产文本、不产真实文件，故结论 inconclusive。

1. 实验过程与结果 / Process & Results

目标 / Goal

验证一个机制，不是刷分：复用已跑通的 AHE 「evolve→falsify→rollback」闭环，把它用到 quant/finance 任务上，看这套自动演化 agent 外挂的闭环能否在 quant 域真实工作并带来提升。

名词速览（给没看过这个实验的人） / Glossary

- **harness (外挂)**: 模型外面那套可改的东西；这里拆成 7 个 slot: `tool / middleware / skill / prompt / validator / memory / router`。
- **quant_agent (worker, 执行器)**: 在当前 harness 下真正去做题、产出"交付物"的 agent。
- **evolve_agent (演化器)**: 读失败诊断，每轮只改 harness 的一个 slot，试图让 worker 做得更好。
- **verifier (验证器)**: 只负责给交付物打分。硬 verifier = 确定性数值核对；软 judge = LLM 按 rubric 打分。
- **falsify / gate (判定 + 去留)**: `falsify.py` 里的一步——把"改前 vs 改后"的分数变化判成 `verdict(EFFECTIVE/.../HARMFUL)`，再由 `gate` 决定这个 edit 留下还是回滚。注意 `gate` 不属于 `verifier` (`verifier` 只打分)；它是 `falsify` 的决策规则：硬模式 = 严格(任何未预测掉分就回滚)，软模式 = `noise-aware`(聚合分超过噪声底才留)。
- **OOS pass**: 一道题"算通过"。

整体流程 / How it runs (end-to-end)

HARNESS = 7 个可演化 slot（模型外面那套可改的"外挂"）：

```
+-----+
| tool*  middleware  skill  prompt  |
| validator  memory  router  |
+-----+
```

```

+-----+
*seed 只填 tool = code_exec, 其余 6 个 slot 空着
  ^  evolve_agent 每轮只改其中 1 个 slot
  |
task ----> quant_agent ----> 交付物 ----> router
                        |-- A 堆 --> HardVerifier (数值重算 + probe)
                        +-- B 堆 --> SoftJudge (rubric -> 0/0.5/1)
                                \--> 每题分数

```

一轮 iteration (闭环的一圈) :

- (1) evaluate 全部任务 -> 分数
- (2) diagnose (ADB-lite) -> root cause
- (3) evolve_agent 读 (1)(2) + rejected-edit buffer -> 提 1 个 edit (改某 slot)
- (4) 套到 harness 副本 -> 重新 evaluate
- (5) falsify: 出 verdict + 过 gate -> keep / rollback(回滚, 入 buffer)
- (6) 落盘 3 层 trace + resume.json (可断点续跑)

设计受 4 条"铁律"约束: ①只打 harness 是真瓶颈的任务; ②硬 verifier 入回路(软信号只做迁移评测); ③多次评估去噪; ④按子类/职业分别记分、不要单一聚合。复用 AHE 的 verdict 引擎/manifest/三层 observability; 移植 SkillOpt 的 rejected-edit buffer + strict gate + edit budget; **quant 新增** integrity guard = perturbation probe (防硬编码)。代码 ~1956 行 / 13 commits / 17 单测全过。

怎么评分 / Scoring (关键, 三套要分清)

- **硬 verifier** (mock + 合成 A 堆): 每题 0 / 0.5 / 1。1=答案对且通用 (扰动输入仍对, 过 perturbation probe); 0.5=base 对但硬编码 (过不了 probe); 0=base 就错。题"通过"= 拿到 1。
- **软 judge** (真实 GDPval): 把该题的 rubric_json 逐条判「满足? 是/否」×该条分值, 得到加权完成度, 再量化到与专家标准 parity 的 {0, 0.5, 1}——0=低于标准 (完成度 <0.5) / 0.5=持平 (0.5—0.8) / 1=达到或超过 (≥0.8, ≈与人工 gold 一样好或更好)。per-occupation 「pass rate」= 这些 {0,0.5,1} 的均值 (即 GDPval 式 win-rate, 平局算半分); 单题"通过"= ≥0.5 (持平或更好)。
- **GDPval 官方评分** (仅作对照, 未采用): 人工/GPT-5 把"模型交付物 vs 人工标准答案"做 pairwise 盲评 → 0 / 0.5 / 1 (模型更好 / 平 / 人更好) → win-rate。无公开 API (只有网页表单且实测无法提交), 所以我们用上面对齐其 0/0.5/1 标准的自建 rubric 评分代替。

注: 下方 E1—E4 的软评分数字采于早期连续 rubric ([0,1] 均值, 阈值 0.6); 评分逻辑已改为上述 {0,0.5,1} parity 制, 重跑会以 win-rate 重新表达 (结论方向不变)。

任务长什么样 / What the tasks are

- **合成 A 堆** (有确定数值答案, 驱动硬回路): Black-Scholes 期权定价、贷款摊销表、流动比率 (audit metric)、NPV+IRR 估值。每个引用一个真实 GDPval task_id 标血缘。

- **真实 GDPval finance** (30 题 = 6 类职业 ×5)：真实职业交付物。例：审计员对一张 Anti-Financial-Crime 指标表做抽样测试、要求交一个含 'Sample Size Calculation' 工作表的 Excel 文件*；理财顾问写配置调整建议备忘录；投资分析师*写估值备忘。

数据 / Data (四组实验)

#	实验	配置	结论先行
E1	Mock 机制验证(离线、硬 verifier)	脚本化、2-arm	闭环成立：四信号全 PASS、17/17 测试、HEADROOM CONFIRMED
E2	合成 A-pile real	deepseek-v4-pro, 4 iter	能力够、无 headroom：seed 4/7, 4 个 edit 全 HARMFUL/INEFFECTIVE→平
E3	GDPval-soft (pro)	pro worker+judge, 30 题, 4 iter	gate 太严：2 个 edit 真抬了分(0.618→0.651/0.645) 却被回滚 → 催生 noise-aware gate
E4	GDPval-soft (flash/pro)	flash worker + pro evolve, 8 iter	被噪声卡住：seed~0.60, 噪声底 0.0276, 0/8 kept → SOFT HEADROOM NOT OBSERVED (inconclusive)

E4 各职业通过率 / per-occupation pass rate (两列估计的差距本身就是噪声证据)：

职业	pass rate(6 次平均, 稳)	pass rate(单次)	mean
Personal Financial Advisors	100%	100%	~0.82
Real Estate Brokers	86.7%	100%	~0.78
Financial & Investment Analysts	66.7%	80%	~0.62
Financial Managers	56.7%	40%	~0.52
Securities/Commodities Sales	43.3%	60%	~0.51
Accountants and Auditors	13.3%	0%	~0.28

mean = 该职业 5 道题 (×多次评估) 的平均 rubric 完成度 (早期连续制 [0,1], 1 = 满足全部 rubric 条目)；pass rate = 完成度过阈值的题占比。按职业分开记 (铁律④)。改成 {0,0.5,1} 制后, per-occupation 指标即「该职业的 win-rate」。

案例研究 / Case Study

Case A — 闭环真的在工作 (mock, 可审计)。结论：四个阶段因果连通、回滚正确。证据：iter1 的诊断说 root cause = Hardcoding (6 题过 base 但过不了 probe) → manifest 的 root_cause 同样写 Hardcoding → workspace 落盘显示 validator slot 新增 integrity_guard → verdict EFFECTIVE、通过数 0→6 (同一根因贯穿四层 json)。随后 iter2 故意改坏 code_exec → 全错 → HARMFUL 正确回滚；iter4 重提该坏 edit → 被 rejected-edit buffer 提前拦下。

Case B — worker 现在交的是"文本"而非真实文件 (real GDPval, 限制所在)。结论：worker 不写 .xlsx/.pptx, 只产一段文本, 所以一大类 rubric 分结构性拿不到。证据：上面那道审计任务要求"交一个含 'Sample Size Calculation' 工作表的 Excel workbook (每条 +2 分) "；实跑 flash worker 返

回的是 `type=str` 的 4864 字 markdown——它把公式 $n=Z^2p(1-p)/e^2$ 、 $n\approx 68$ 、"A1—A4 放输入、加一列 QoQ Variance"等用文字描述出来，但没有任何真实文件。而且代码里只把 `task.prompt` 喂给它、没喂 **reference** 附件表，所以它连真实数据都拿不到、只能自己编个例子 ($N=500$)。→ "是 Excel 文件 / 有名为 X 的 sheet / 用附件数据算出的值"这些条目**必挂**，这正是 Accountants/Auditors 仅 ~0—13% 的直接原因（这类职业最依赖真文件+附件数据）。

Case C — edit 抬了分却被噪声回滚 (real GDPval, falsification 的核心两难)。 结论：**evolve_agent** 提的 edit 真把聚合分抬了，却被判成"可能过拟合/副作用"而回滚——这是软信号下 falsification 最棘手的地方。证据(E3): `skill:financial_computation_skill` 把均分 0.618→**0.651**、`middleware:variable_pay_middleware` →**0.645**，但都拿到 verdict MIXED → 回滚、最终 0 kept。

– 回滚的判定理由（是不是过拟合？）：旧 strict gate 看到该 edit 造成了未预测的 task 掉分 (unattributed regression)，按设计把任何未预测掉分当作潜在的过拟合/副作用伤害 (edit 偏帮了它声明要修的任务、却伤了别的任务) → 保守回滚。

– 但真因多半是噪声、不是过拟合：软 judge 每轮重新生成交付物 + 重新打分，同一 harness 也会有 1—2 个 task 随机掉分。单样本下"随机掉分"和"真过拟合掉分"无法区分，于是净正的 edit 被误杀。

– 怎么解决：① **noise-aware gate** (已实现) ——不再"有掉分就回滚"，改为"聚合分超过噪声底才 keep"，容忍少量随机掉分；② 对 worker 交付物 k 次采样取中位——让 per-task 分稳定，真过拟合（系统性掉分）就能与噪声（随机掉分）区分；③ 引入 held-out / selection split (v0 暂未做) ——真过拟合会在 held-out 上掉、噪声不会，这是区分"过拟合 vs 噪声"的正解。E4 已用 ① 但仍 0/8，说明单靠 noise-aware gate 不够，② 是下一步头号动作。

2. 分析 / Analysis（结论先行）

- 机制成立、且忠实复现 —— 因为 E1 在干净硬信号下把四条信号全点亮(含可审计因果链)，E2—E4 又证明回路能在**真实 LLM + 真实 GDPval** 上端到端跑通。
- "无 headroom"有两种不同的脸 —— (a) **能力够型**(E2): deepseek 把规范的数值题做得很好 (BS 精确、IRR 收敛到 $1e-13$)，harness 无从发力；(b) **噪声受限型**(E3/E4): 软评分每轮重生成交付物+重打分，必然抖动，falsification **分不清** edit 真实效果。这把"软信号入回路"的代价实测了出来。
- **E3→E4 的演化说明 gate 设计很关键** —— E3 旧 gate 「任何掉分就回滚」误杀了净正 edit；改成 noise-aware gate(只认超过噪声底的聚合提升)后,E4 又显示**单样本基线+软噪声**仍盖过 edit 效应(8 个 delta 全在 ± 0.0276 内)。所以"0/8 kept"**不等于**"evolve 没用",而是"信号分辨率不够"。
- **per-occupation 比单一聚合分有用** —— Accountants/Auditors 在两种估计下都垫底(~0—13%)、Advisors/Real-Estate 接近天花板，原因清楚:前者要求真 Excel+附件计算(worker 都没有)，后者交付物本就偏叙述文本(纯文本恰好够用)。这印证了铁律④。

3. 问题与困难（待讨论） / Problems & Open Questions

已解决的工程坑（都已修+commit）：sandbox 挡掉 `import math`；IRR 容差套错($\pm 643\%$ 放水)；未归因 regression 混过 EFFECTIVE；2.4 小时死锁(client 无超时+SOCKS 代理 stall) → 加 90s 超时 + 并发 + 单任务降级；`--resume` + 每轮 checkpoint。

待你拍板的开放问题（先说要决定什么，再说背景）：

1. 论文 novelty 锚在哪（研究定位，最该先想清楚）—— 要决定：贡献点立在「演化搜索的时间/样本复杂度优化」上，还是「evolving-agent 框架本身」上，还是两者组合。背景：讨论过可从搜索的时间复杂度入手优化（用更少的 verifier 调用就找到好 edit —— 对应 ROADMAP 的 prioritized/credit-assigned search、multi-fidelity verifier、*fitness vs verifier-call-budget* 曲线）；但那只是“搜索效率”，真正的差异化仍需要一个 evolving-agent 框架——harness 级演化 + 硬 verifier 落到 quant 非平稳域，这是现有工作没碰的点。倾向：novelty 锚在「框架」上，把复杂度优化做成量化支撑实验（对照 Life-Harness 全迭代 / AHE 文件编辑 baseline）。先定位再补实验。
2. 去噪 vs 成本（头号技术阻塞）—— 要让软回路能定论，得把 eval 噪声压下去；最直接是对 worker 交付物 k 次采样取中位。tradeoff:eval 调用翻 k 倍(flash 便宜，k=3 可接受)。上不上？
3. 软信号到底值不值得继续 —— GDPval 无硬 verifier、用软 rubric 驱动是主动放松铁律②。继续在软信号上去噪，还是换一个自带硬 verifier 的 family(如 FinRL-Meta 摩擦回测)拿干净闭环？
4. judge 偏置 —— 现在是 deepseek 自评 rubric(偏宽松、循环偏置)，官方 grader 无 API。要不要换更强/中立的 judge 做校准？

4. 下周计划 / Next Week's Plan（按优先级）

1. worker 交付物 k-sample 去噪(#1 阻塞)：每个交付物生成 k=3 次取 median，稳住基线，重跑 flash/pro → 拿到能定论的 soft-headroom 判定。
2. 真实文件生成 + 喂 reference 附件：worker 改为写代码产 .xlsx/.pptx；机械/格式条目走确定性硬检查 → 抬升文本下限、部分找回铁律②。
3. 硬 verifier family 对照：接 FinRL-Meta(带摩擦回测、防泄漏) 做一个真硬信号闭环，与软 GDPval 对照,直接量出“软信号拖累了多少”。
4. judge 升级/校准：复现版 pairwise-vs-gold judge 指向更强模型;周期性手动提交官方 grader 做 calibration。

依赖：#1—#4 需联网 + OpenRouter 余额；官方 grader 校准被“无 API”阻塞，只能人工网页提交。